

1 裁剪算法和多边形扫描转换实验

1.1 实验目的

1. 熟悉 Cohen-sutherland 算法和 Liang-Barsky 算法的思想，并掌握至少一种裁剪算法的实现
2. 熟悉 X-扫描线算法和边标志算法的思想，并掌握至少一种多边形扫描转换的实现
3. 掌握一定的算法分析和优化的能力

1.2 实验要求

```
class MyAlgorithm : public ClipAlgorithm
{
    //可以在这里定义成员变量和成员函数
public:
    static MyAlgorithm* CreateMyAlgorithm()
    {
        return new MyAlgorithm();
    }
    //如果最终不保留线段，直接返回false
    //否则，将p1, p2更新为最终保留线段的两端点（无顺序要求，误差在±0.01以内），然后返回true
    bool Clip(float xMin, float xMax, float yMin, float yMax, Vector2& p1, Vector2& p2) override
    {
        return false; //完成这个函数
    }
};
```

1. 完成实验项目中如上图所示的函数，以实现裁剪算法，并回答以下问题：

- 1) 用自然语言简述你如何实现这一函数。

本实验在矩形裁剪窗口 $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$ 下，实现了两个线段裁剪函数：MyAlgorithm::Clip（自定义方法）与 LiangBarskyImprovedAlgorithm::Clip（改进 Liang-Barsky 方法）。二者输入均为窗口边界与线段两端点，输出为是否保留线段及裁剪后的新端点。

MyAlgorithm::Clip 的实现

设计思路

MyAlgorithm::Clip 采用“快速排斥 + 特殊情形分支 + 端点逐边修正”的策略：

1. **快速排斥**：若两端点同时位于窗口同一外侧（左、右、下、上任一方向），则线段必不可见，直接返回 false。
2. **特殊情形处理**：
 - 当 $(dx \approx 0)$ 近似竖直线时，避免斜率计算中的除零问题，仅对 y 方向进行截断；
 - 当 $(dy \approx 0)$ 近似水平线时，仅对 x 方向进行截断。
3. **一般斜线处理**：对两个端点分别执行边界修正：
 - 端点超出左右边界时，先固定 x，再由直线方程反算 y；
 - 端点超出上下边界时，先固定 y，再反算 x。
4. **结果复检**：若修正后线段仍整体在窗口外，则舍弃；否则更新端点并返回 true。

关键公式

设原线段端点为 (x_1, y_1) 、 (x_2, y_2) ，有 $dx = x_2 - x_1, dy = y_2 - y_1$

- 已知 $x=x_b$ （与左右边相交）时： $y = y_1 + \frac{dy}{dx}(x_b - x_1)$
- 已知 $y=y_b$ （与上下边相交）时： $x = x_1 + \frac{dx}{dy}(y_b - y_1)$

程序中通过阈值 ε 判断 (dx, dy) 是否接近 0，以增强数值稳定性。

正确性说明

该方法先通过快速排斥保证必不可见线段被及时剔除，再利用边界约束将端点逐步投影到可见区域边界。

对竖直/水平线的单独分支避免了除零与不稳定计算。最终的合法性复检确保返回结果满足窗口约束，因此算法在几何上是正确的。

时间复杂度

窗口边界数量固定为 4，端点数量固定为 2，所有操作为常数次计算，因此时间复杂度为 $O(1)$ ，空间复杂度为 $O(1)$ 。

LiangBarskyImprovedAlgorithm::Clip 的实现

设计思路

该函数基于 Liang-Barsky 参数裁剪思想。将线段表示为 $P(t) = P_0 + t(P_1 - P_0)$ ， $t \in [0, 1]$ ，通过更新可见参数区间 $[t_0, t_1]$ 来得到裁剪结果。主要步骤如下：

1. **快速排斥**：与前一方法一致，先剔除明显不可见线段。
2. **参数区间初始化**： $t_0=0$ ， $t_1=1$ 。
3. **定义 clip_t(p, q)**：
 - $p=0$ ：线段与某边平行，若 $q<0$ 则不可见；
 - $p<0$ ：进入边，可能增大 t_0 ；
 - $p>0$ ：离开边，可能减小 t_1 。
4. **按需激活边界测试**：仅当端点在某方向上可能越界时，才调用对应边的 clip_t，减少无效比较与除法。
5. **由最终参数重建端点**：若 $t_0 \leq t_1$ ，则计算新端点；否则返回不可见。

参数更新含义

每条边会给出一个关于 t 的不等式约束。不断求交后，可得到最终可见区间 $[t_0, t_1]$ ：

- 当新区间为空（ $t_0 > t_1$ ）时，线段与窗口无交；
- 当新区间非空时，线段在该区间对应的部分即为裁剪结果。

正确性说明

Liang-Barsky 的本质是把二维几何裁剪转换为一维参数区间求交。每条边界都对应对 t 的约束，全部约束同时满足即得到可见段。本实现保留了该理论框架，并加入按需边界检测优化，不改变几何意义，因此结果仍然正确。

时间复杂度

边界数固定为 4，参数更新为常数次运算，复杂度同样为 $O(1)$ 空间复杂度 $O(1)$ 。

2) 从你实现的函数出发，列出一个影响算法性能的点并简要分析。

MyAlgorithm::Clip 相对 Cohen-Sutherland 的优点：减少区域编码与反复迭代开销

Cohen-Sutherland 依赖端点区域码并进行多轮“取外点-求交-更新-重编码”的循环。我的 MyAlgorithm::Clip 采用“快速排斥 + 特殊情形直裁 + 端点边界修正”，在常见场景下能更快进入有效分支，减少位编码更新和循环迭代带来的额外操作。两者理论复杂度均为 $O(1)$ ，但在批量测试中，MyAlgorithm 的实现路径更直接，常数项通常更小，执行时间更稳定。

LiangBarskyImprovedAlgorithm::Clip 相对基础 Liang-Barsky 的优点：按需边界测试降低无效计算

基础 Liang-Barsky 通常对四条边都执行参数裁剪；我实现的改进版本先判断线段是否可能在该方向越界，仅在必要时调用 clip_t 更新 t。该策略减少了不必要的比较与除法运算，尤其在线段大多位于窗口内或仅与少数边相关时，能有效降低常数开销，因此在多次重复实验中通常更快。

我写的函数还有可能改进的点：

MyAlgorithm::Clip 性能点：分支判断与重复交点计算较多

该实现先做快速排斥，再区分竖直/水平/一般情况，并在一般情况下对两个端点逐次做越界修正。分支路径较多（if/else 层次深），且在端点修正时会多次做除法与乘法。虽然单次仍是 $O(1)$ ，但常数项偏大；在高频批量裁剪（例如循环十万次）时，分支预测失败和重复浮点运算会拉低吞吐。

运行截图如下：

```

CohenSutherland算法：
运行10000000次测试用例花费415毫秒

LiangBarsky算法：
运行10000000次测试用例花费261毫秒

我的裁剪算法：
输入：xMin=10,xMax=20,yMin=10,yMax=20,p1(0,0),p2(30,30)
答案：(10,10) (20,20)
输出：(10,10) (20,20)
输出正确

输入：xMin=10,xMax=20,yMin=10,yMax=20,p1(5,8),p2(15,14)
答案：(10,11) (15,14)
输出：(10,11) (15,14)
输出正确

输入：xMin=10,xMax=20,yMin=10,yMax=20,p1(8,25),p2(23,19)
答案：(不保留线段)
输出：(不保留线段)
输出正确

运行这100个测试用例的准确率为100.00%
运行10000000次测试用例花费240毫秒

LiangBarsky改进算法(CSD-92-688)：
输入：xMin=10,xMax=20,yMin=10,yMax=20,p1(0,0),p2(30,30)
答案：(10,10) (20,20)
输出：(10,10) (20,20)
输出正确

输入：xMin=10,xMax=20,yMin=10,yMax=20,p1(5,8),p2(15,14)
答案：(10,11) (15,14)
输出：(10,11) (15,14)
输出正确

输入：xMin=10,xMax=20,yMin=10,yMax=20,p1(8,25),p2(23,19)
答案：(不保留线段)
输出：(不保留线段)
输出正确

运行这100个测试用例的准确率为100.00%
运行10000000次测试用例花费250毫秒

```

可见我的 cohen 改进算法有显著提升，而 liang 算法提升不大，也许是 Liang 算法本身性能就足够优越。

源代码如下：

program.cpp

```

1. #include "Program.h"
2. using namespace std;
3.
4. class MyAlgorithm : public ClipAlgorithm
5. {
6.     //可以在这里定义成员变量和成员函数
7. public:
8.     static MyAlgorithm* CreateMyAlgorithm()
9.     {
10.         return new MyAlgorithm();
11.     }
12.     //如果最终不保留线段，直接返回 false
13.     //否则，将 p1, p2 更新为最终保留线段的两端点（无顺序要求，误差在±0.01 以内），然后返回 true

```

```
14.     bool Clip(float xMin, float xMax, float yMin, float yMax, Vector2
      & p1, Vector2& p2) override
15.     {
16.         float x1 = p1.x;
17.         float y1 = p1.y;
18.         float x2 = p2.x;
19.         float y2 = p2.y;
20.         float dx = x2 - x1;
21.         float dy = y2 - y1;
22.         const float eps = 1e-6f;
23.
24.         if ((x1 < xMin && x2 < xMin) || (x1 > xMax && x2 > xMax) ||
25.             (y1 < yMin && y2 < yMin) || (y1 > yMax && y2 > yMax))
26.             return false;
27.
28.         if (dx > -eps && dx < eps)
29.         {
30.             if (x1 < xMin || x1 > xMax) return false;
31.             if (y1 < yMin) y1 = yMin;
32.             else if (y1 > yMax) y1 = yMax;
33.             if (y2 < yMin) y2 = yMin;
34.             else if (y2 > yMax) y2 = yMax;
35.             p1 = Vector2(x1, y1);
36.             p2 = Vector2(x2, y2);
37.             return true;
38.         }
39.
40.         if (dy > -eps && dy < eps)
41.         {
42.             if (y1 < yMin || y1 > yMax) return false;
43.             if (x1 < xMin) x1 = xMin;
44.             else if (x1 > xMax) x1 = xMax;
45.             if (x2 < xMin) x2 = xMin;
46.             else if (x2 > xMax) x2 = xMax;
47.             p1 = Vector2(x1, y1);
48.             p2 = Vector2(x2, y2);
49.             return true;
50.         }
51.
52.         float clippedX[2] = { x1, x2 };
53.         float clippedY[2] = { y1, y2 };
54.
55.         for (int i = 0; i < 2; ++i)
56.         {
```

```

57.         if (clippedX[i] < xMin)
58.         {
59.             clippedX[i] = xMin;
60.             clippedY[i] = (dy / dx) * (xMin - x1) + y1;
61.         }
62.         else if (clippedX[i] > xMax)
63.         {
64.             clippedX[i] = xMax;
65.             clippedY[i] = (dy / dx) * (xMax - x1) + y1;
66.         }
67.
68.         if (clippedY[i] < yMin)
69.         {
70.             clippedY[i] = yMin;
71.             clippedX[i] = (dx / dy) * (yMin - y1) + x1;
72.         }
73.         else if (clippedY[i] > yMax)
74.         {
75.             clippedY[i] = yMax;
76.             clippedX[i] = (dx / dy) * (yMax - y1) + x1;
77.         }
78.     }
79.
80.     if ((clippedX[0] < xMin && clippedX[1] < xMin) || (clippedX[0]
81.         ] > xMax && clippedX[1] > xMax) ||
82.         (clippedY[0] < yMin && clippedY[1] < yMin) || (clippedY[0]
83.         ] > yMax && clippedY[1] > yMax))
84.         return false;
85.
86.     p1 = Vector2(clippedX[0], clippedY[0]);
87.     p2 = Vector2(clippedX[1], clippedY[1]);
88.     return true;
89. }
90. };
91.
92. class LiangBarskyImprovedAlgorithm : public ClipAlgorithm
93. {
94. public:
95.     static LiangBarskyImprovedAlgorithm* CreateLiangBarskyImprovedAl
96.         gorithm()
97.     {
98.         return new LiangBarskyImprovedAlgorithm();
99.     }

```

```

98.     bool Clip(float xMin, float xMax, float yMin, float yMax, Vector2
      & p1, Vector2& p2) override
99.     {
100.         float x0 = p1.x, y0 = p1.y;
101.         float x1 = p2.x, y1 = p2.y;
102.
103.         // 1. Trivial reject
104.         if ((x0 < xMin && x1 < xMin) ||
105.             (x0 > xMax && x1 > xMax) ||
106.             (y0 < yMin && y1 < yMin) ||
107.             (y0 > yMax && y1 > yMax))
108.             return false;
109.
110.         float dx = x1 - x0;
111.         float dy = y1 - y0;
112.
113.         float t0 = 0.0f;
114.         float t1 = 1.0f;
115.
116.         auto clipt = [&](float p, float q) -> bool
117.         {
118.             // 平行情况（避免除 0）
119.             if (p == 0.0f)
120.                 return q >= 0.0f;
121.
122.             if (p < 0.0f)
123.             {
124.                 // entering: q <= p * t1
125.                 if (q < p * t1) return false;
126.                 if (q < p * t0) t0 = q / p;
127.             }
128.             else
129.             {
130.                 // leaving: q >= p * t0
131.                 if (q < p * t0) return false;
132.                 if (q < p * t1) t1 = q / p;
133.             }
134.             return t0 <= t1;
135.         };
136.
137.         // 2. 按需激活边界，避免无关计算
138.         if ((x0 < xMin || x1 < xMin) && !clipt(-dx, x0 - xMin)) re
turn false; // 左

```

```

139.         if ((x0 > xMax || x1 > xMax) && !clipt(dx, xMax - x0)) return false; // 右
140.         if ((y0 < yMin || y1 < yMin) && !clipt(-dy, y0 - yMin)) return false; // 下
141.         if ((y0 > yMax || y1 > yMax) && !clipt(dy, yMax - y0)) return false; // 上
142.
143.         // 3. 计算裁剪结果
144.         if (t1 < 1.0f)
145.         {
146.             x1 = x0 + t1 * dx;
147.             y1 = y0 + t1 * dy;
148.         }
149.         if (t0 > 0.0f)
150.         {
151.             x0 = x0 + t0 * dx;
152.             y0 = y0 + t0 * dy;
153.         }
154.
155.         p1 = Vector2(x0, y0);
156.         p2 = Vector2(x1, y1);
157.         return true;
158.     }
159. };
160.
161. static void TestMyAlgorithm(std::string path, int times)
162. {
163.     std::cout << "我的裁剪算法:" << std::endl;
164.     Test(path, times, 3, MyAlgorithm::CreateMyAlgorithm);
165. }
166.
167. static void TestLiangBarskyImprovedAlgorithm(std::string path, int times)
168. {
169.     std::cout << "LiangBarsky 改进算法:" << std::endl;
170.     Test(path, times, 3, LiangBarskyImprovedAlgorithm::CreateLiangBarskyImprovedAlgorithm);
171. }
172.
173.
174. //注意在 Release 而不是 Debug 模式下运行此程序
175. int main(int argc, char* argv[])
176. {
177.     string path = argv[0];

```



```
178.     path = path.substr(0, path.find("\\x64\\") + "\\Answer.txt";
179.     int times = 100000;    //控制算法运行次数; 测试集中包含 100 个测试
    用例, 则总运行次数=times×100
180.
181.     system("title 裁剪算法");
182.     TestCohenSutherlandAlgorithm(path, times);
183.     TestLiangBarskyAlgorithm(path, times);
184.     TestMyAlgorithm(path, times);
185.     TestLiangBarskyImprovedAlgorithm(path, times);
186.
187.     system("pause");
188. }
```

```

class MyAlgorithm : public ScanConversingAlgorithm
{
    //可以在这里定义成员变量和成员函数
public:
    static MyAlgorithm* CreateMyAlgorithm()
    {
        return new MyAlgorithm();
    }

    //buffer相当于颜色缓冲,左下角片元坐标为(0,0), 右上角片元坐标为(width-1,height-1)
    //根据输入的vertices,更新buffer,不在多边形内部的点保持默认值0, 在多边形内部的点应设为1
    //注意填色时的边界问题,本项目遵循左闭右开, 下闭上开的规则
    void Scan(Vector2Int* vertices, int vertexCount, int** buffer, int width, int height) override
    {
        //完成这个函数
    }
};

```

2. 完成实验项目中如上图所示的函数，以实现多边形扫描转换算法，并回答以下问题：

1) 用自然语言简述你如何实现这一函数。

我们自定义的扫描转换函数总体上采用了与 X 扫描线算法相同的核心框架，即基于有序边表（ET）和活性边表（AET）逐扫描线填充。实现时先由顶点构建边表，再逐行上移扫描线；每一行更新活性边集合后，按交点 x 坐标排序结果进行两两配对，得到填充区间 $[xLeft, xRight)$ ，并将区间内像素写为 1。与参考 X 扫描线算法相比，我们的实现去除了中间交点数组的往返构造，直接从活性边集合配对填充，同时采用了更轻量的边维护方式，减少了扫描过程中的额外开销。

与边标志算法相比，我们的方法在思想上不同：边标志算法是先离散边并在行内通过标志翻转决定内外，而我们是通过实时维护交点来直接生成填充区间；两者都遵循“左闭右开、下闭上开”的边界规则，但数据组织与每行计算路径不同。整体上，三种算法都能得到正确填充结果，差异主要体现在扫描线推进时的状态维护方式和运行开销分布。

2) 从你实现的函数出发，列出一个影响算法性能的点并简要分析。

我选择按当前路径优化，核心原因是测试场景属于高重复、小规模、热点集中的循环计算：同一批用例会被反复执行（times=10000），因此单次扫描中哪怕很小的常数开销，也会被放大成明显的总耗时差异。基于这一特点，我优先优化了最频繁路径上的操作：一是修复边表内存释放，避免长时间运行时因泄漏导致性能退化；二是减少容器高成本操作（如循环内多次 erase、每行全量排序），改为更贴合扫描线“逐行渐变”特性的增量维护；三是去掉中间交点数组，降低不必要的搬运；四是简化交点取整的高频函数调用，降低数值转换开销。总体来说，这些优化都围绕同一个目标：不改变算法结果与边界规则的前提下，压缩热点路径常数成本，提高总运行效率与稳定性。

具体的性能优化如下：

1. 内存管理完善

我们为 `OrderedEdgeContainer` 补充了析构逻辑，统一释放边表桶数组及链表边节点，解决了高次数测试下的内存泄漏问题。该优化不直接改变单次耗时，但显著提升了长时

间运行的稳定性和整体性能一致性。

2. 活性边删除过程优化

在 `MoveUp` 中，将原先循环内多次 `erase` 的方式改为一次 `remove_if + erase`。这样避免了重复移动元素造成的额外开销，降低了每条扫描线的边维护成本。

3. 交点中间数组开销消除

我们在自定义算法中去掉了逐行构建 `xs` 交点数组的流程，改为直接从活性边集合中按顺序两两配对得到填充区间。该优化减少了容器写入、清空和额外遍历，简化了热点路径。

4. 活性边排序策略优化

将“每扫描线全量 `std::sort`”改为“增量有序维护”思路：对已激活边做局部重排，对新进入边按有序位置插入。与全量排序相比，这种方式更符合扫描线逐步变化的特点，降低了排序部分的常数开销。

5. 交点取整开销优化

将 `CurrentX()` 中的 `lroundf` 替换为更轻量的分支取整表达式（ $\pm 0.5f$ 后转整型），降低了高频交点计算的函数开销，同时保持了原有取整语义。

6. 与参考算法的关系

我们的算法框架与 `X` 扫描线算法同属 `ET + AET`，但通过减少中间数据和优化边维护，降低了实现常数。边标志算法属于另一条实现路线（边离散+标志翻转），结构更简单，因此在小分辨率测试中仍可能表现更快；本次优化后，两类算法都因公共基础组件改进而获得提升。

总体而言，本次优化覆盖了内存管理、容器操作、排序策略与数值计算等关键路径，使算法在高重复测试场景下具备更好的效率与稳定性。

`X` 扫描算法和边标志算法耗时如下：

运行1000000次测试用例花费620毫秒
边标志算法：
运行1000000次测试用例花费364毫秒

我们的算法耗时如下：

输出正确
运行这100个测试用例的准确率为100.00%
运行1000000次测试用例花费405毫秒

从最终耗时看，`X` 扫描线算法耗时最长，我们的算法相比参考算法有所优化，这说明：我们的优化有效降低了 `ET+AET` 路线的开销，值得注意的是边标志算法在优化后也得到了加

速并且耗时最短。分析原因，可能主要有三点：

1. 算法路径常数差异

边标志算法不需要维护活性边表排序，也不需要交点配对结构维护，核心是“边离散 + 行内翻转”，每行流程更短、分支更少，常数项天然更低。

2. 测试规模偏小

在小分辨率下，复杂数据结构的理论优势不明显，反而实现开销主导性能。边标志这种简单循环更容易跑出极低时间。

3. 边标志也被“连带优化”了

你们把 `OrderedEdge::CurrentX()` 从 `lroundf` 改为轻量取整，这一改动是公共基础组件优化，边标志算法在 `Discrete()` 中同样大量调用 `CurrentX()`，因此它也获得了显著提速。

此外，前面修复了边表相关内存释放后，长次数测试下整体内存压力更稳定，也会间接改善后续算法的测试表现。

源代码如下：

Program.cpp

```
1. #include "Program.h"
2. #include<random>
3. using namespace std;
4.
5. class MyAlgorithm : public ScanConversingAlgorithm
6. {
7.     //可以在这里定义成员变量和成员函数
8. public:
9.     static MyAlgorithm* CreateMyAlgorithm()
10.    {
11.        return new MyAlgorithm();
12.    }
13.
14.    //buffer 相当于颜色缓冲,左下角片元坐标为(0,0)，右上角片元坐标为
        (width-1,height-1)
15.    //根据输入的 vertices,更新 buffer,不在多边形内部的点保持默认值 0，在多
        边形内部的点应设为 1
16.    //注意填色时的边界问题，本项目遵循左闭右开，下闭上开的规则
17.    void Scan(Vector2Int* vertices, int vertexCount, int** buffer, in
        t width, int height) override
18.    {
19.        OrderedEdgeContainer orderedEdges(vertices, vertexCount);
20.        ActiveEdgeContainer activeEdges(orderedEdges);
21.        for (; activeEdges.MoveUp(); )
22.        {
23.            for (size_t i = 0; i + 1 < activeEdges.edges.size(); i += 2)
```

```

24.         {
25.             int xLeft = activeEdges.edges[i]->CurrentX();
26.             int xRight = activeEdges.edges[i + 1]->CurrentX();
27.             for (int j = xLeft; j < xRight; j++)
28.             {
29.                 buffer[j][activeEdges.currentY] = 1;
30.             }
31.         }
32.     }
33. }
34. };
35.
36. static void TestMyAlgorithm(std::string path, int times)
37. {
38.     std::cout << "我的扫描转换算法:" << std::endl;
39.     Test(path, times, 3, MyAlgorithm::CreateMyAlgorithm);
40. }
41.
42. int main(int argc, char* argv[])
43. {
44.     string path = argv[0];
45.     path = path.substr(0, path.find("\\x64\\")) + "\\Answer.txt";
46.     int times = 10000;    //控制算法运行次数: 测试集中包含 100 个测试用例, 则总运行次数=times×100
47.
48.     ConsoleUtility::DisableWindowResize();
49.     system("title 扫描转换算法");
50.     TestXScanlineAlgorithm(path, times);
51.     TestEdgeFlagAlgorithm(path, times);
52.     TestMyAlgorithm(path, times);
53.     system("pause");
54. }

```

ActiveEdgeContainer.h

```

1. #pragma once
2. #include<vector>
3. #include"OrderedEdge.h"
4. #include"OrderedEdgeContainer.h"
5.
6. class ActiveEdgeContainer
7. {
8. public:
9.     std::vector<OrderedEdge*> edges;
10.    //this class do not takes ownership of container

```

```

11.     OrderedEdgeContainer& container;
12.     int currentY;
13.
14.     ActiveEdgeContainer(OrderedEdgeContainer& container);
15.     bool MoveUp();
16.     bool MoveUp(std::vector<int>& ret);
17. };

```

ActiveEdgeContainer.cpp

```

1. #include "ActiveEdgeContainer.h"
2. #include<algorithm>
3.
4. ActiveEdgeContainer::ActiveEdgeContainer(OrderedEdgeContainer& containe
    r)
5.     :container(container)
6. {
7.     currentY = container.yMin - 1;
8. }
9.
10. bool ActiveEdgeContainer::MoveUp()
11. {
12.     currentY++;
13.     edges.erase(std::remove_if(edges.begin(), edges.end(), [this](Ord
        eredEdge* edge)
14.         {
15.             return currentY > edge->yMax;
16.         }), edges.end());
17.
18.     for (int i = 0; i < edges.size(); i++)
19.     {
20.         edges[i]->MoveUp();
21.     }
22.
23.     for (int i = 1; i < edges.size(); i++)
24.     {
25.         OrderedEdge* key = edges[i];
26.         int j = i - 1;
27.         while (j >= 0 && OrderedEdge::Compare(key, edges[j]))
28.         {
29.             edges[j + 1] = edges[j];
30.             j--;
31.         }
32.         edges[j + 1] = key;
33.     }

```

```

34.
35.     OrderedEdge* p = container.GetHead(currentY);
36.     for (; p; p = p->next)
37.     {
38.         p->MoveUp();
39.         auto pos = std::upper_bound(edges.begin(), edges.end(), p, Or
d
eredEdge::Compare);
40.         edges.insert(pos, p);
41.     }
42.
43.     if (edges.size() == 0)
44.         return false;
45.
46.     return true;
47. }
48.
49. bool ActiveEdgeContainer::MoveUp(std::vector<int>& ret)
50. {
51.     if (!MoveUp())
52.     {
53.         ret.clear();
54.         return false;
55.     }
56.
57.     ret.clear();
58.     ret.reserve(edges.size());
59.     for (int i = 0; i < edges.size(); i++)
60.     {
61.         ret.push_back(edges[i]->CurrentX());
62.     }
63.     return true;
64. }

```

OrderedEdgeContainer.h

```

1. #pragma once
2. #include "Vector2Int.h"
3. #include "OrderedEdge.h"
4.
5. class OrderedEdgeContainer
6. {
7. public:
8.     int yMin, yMax;
9.     OrderedEdge** list;
10.     OrderedEdgeContainer(Vector2Int* vertices, int vertexCount);

```

```

11. ~OrderedEdgeContainer();
12. OrderedEdge* GetHead(int y) const;
13. OrderedEdge** GetHeadPtr(int y) const;
14. void Add(OrderedEdge* edge) const;
15. };

```

OrderedEdgeContainer.cpp

```

1. #include "OrderedEdgeContainer.h"
2. using namespace std;
3.
4. OrderedEdgeContainer::OrderedEdgeContainer(Vector2Int* vertices, int vertexCount)
5. {
6.     yMin = INT_MAX;
7.     yMax = INT_MIN;
8.     Vector2Int from, to;
9.     for (int i = 0; i < vertexCount; i++)
10.    {
11.        yMin = min(yMin, vertices[i].y);
12.        yMax = max(yMax, vertices[i].y);
13.    }
14.    int size = yMax - yMin + 1;
15.    list = new OrderedEdge * [size];
16.    for (int i = 0; i < size; i++)
17.    {
18.        list[i] = nullptr;
19.    }
20.    for (int i = 0; i < vertexCount; i++)
21.    {
22.        from = vertices[i];
23.        to = vertices[(i + 1) % vertexCount];
24.        OrderedEdge* edge = OrderedEdge::TryCreateOrderedEdge(from, to);
25.        if (edge)
26.            Add(edge);
27.    }
28. }
29.
30. OrderedEdgeContainer::~OrderedEdgeContainer()
31. {
32.     if (!list)
33.         return;
34.
35.     const int size = yMax - yMin + 1;

```



```

36.     for (int i = 0; i < size; i++)
37.     {
38.         OrderedEdge* p = list[i];
39.         while (p)
40.         {
41.             OrderedEdge* next = p->next;
42.             delete p;
43.             p = next;
44.         }
45.     }
46.     delete[] list;
47. }
48.
49. OrderedEdge* OrderedEdgeContainer::GetHead(int y) const
50. {
51.     return list[y - yMin];
52. }
53.
54. OrderedEdge** OrderedEdgeContainer::GetHeadPtr(int y) const
55. {
56.     return list + (y - yMin);
57. }
58.
59. void OrderedEdgeContainer::Add(OrderedEdge* edge) const
60. {
61.     OrderedEdge** headPtr = GetHeadPtr(edge->yMin);
62.     edge->next = *headPtr; //insert to head
63.     *headPtr = edge;
64. }

```

OrderedEdge.cpp

```

1. #include "OrderedEdge.h"
2.
3. bool OrderedEdge::Compare(OrderedEdge* x, OrderedEdge* y)
4. {
5.     if (x->currentX < y->currentX)
6.         return true;
7.     if (x->currentX > y->currentX)
8.         return false;
9.     if (x->deltaX < y->deltaX)
10.        return true;
11.    return false;
12. }

```

```
13. OrderedEdge* OrderedEdge::TryCreateOrderedEdge(Vector2Int& a, Vector2
    Int& b)
14. {
15.     if (a.y == b.y)
16.         return nullptr;
17.     if (a.y > b.y)
18.     {
19.         Vector2Int temp = a;
20.         a = b;
21.         b = temp;
22.     }
23.     return new OrderedEdge(a, b);
24. }
25. int OrderedEdge::CurrentX() const
26. {
27.     return currentX >= 0.0f
28.         ? static_cast<int>(currentX + 0.5f)
29.         : static_cast<int>(currentX - 0.5f);
30. }
31. OrderedEdge::OrderedEdge(Vector2Int from, Vector2Int to)
32.     :yMin(from.y), yMax(to.y - 1), next(nullptr)    //[down.up)
33. {
34.     deltaX = (to.x - from.x) / (float)(to.y - from.y);
35.     currentX = deltaX * (yMin - from.y - 1) + from.x;
36. }
37. void OrderedEdge::MoveUp()
38. {
39.     currentX += deltaX;
40. }
```

1.3 实验小结

本次的两个实验对应图形管线里最基础、最常见的两类问题，扫描转换解决“几何图元如何离散到像素网格”的问题，裁剪解决“哪些几何内容应该进入后续绘制”的问题。通过这两类实验，可以把课本里的几何公式、边界规则和数据结构真正落到代码层面。尤其是在实现过程中，会直观看到同样的图形结果，不同算法在效率、稳定性和边界一致性上差异很大。这有助于建立正确性、性能与工程性三者统一的图形学思维。

在优化时我最大的心得体会就是不应全靠直觉或者盲目提问 AI，而应自己先定位到瓶颈，明确热点在排序、内存、取整还是写入，再查资料对照：针对热点定向检索关键词，看经典做法与现代实现，在这个过程中可以借助 AI 提升检索效率，这才是正确的应用先进科技首单。既能吸收外部知识（论文、开源实现、AI 建议），又能保证优化结论来自本地实验数据，最终形成可复现、可解释的工程改进过程。

2 图形渲染综合实验

2.1 实验目的

1. 熟悉 OpenGL 中的渲染流程
2. 掌握一些 OpenGL API 的用法
3. 了解一些渲染算法的具体实现

2.2 实验要求

1. 参考所给的 7 个实验项目，在一个场景中，自行选择一些渲染算法来实现，确保这些算法能够协同运行；并回答以下问题：

1) 简述你对 OpenGL 中，VAO, VBO, EBO 的理解。

VAO、VBO、EBO 可以理解为 OpenGL 绘制流程里“数据组织—数据存储—索引复用”的三层机制，它们共同决定 GPU 如何读取顶点并组成图元。简单来说，VBO 存顶点原始数据，EBO 存顶点索引顺序，VAO 存这些数据该怎么解释和绑定的状态。

在本实验框架里，VBO 主要承载位置、法线、纹理坐标、切线等顶点属性数组。它本质是显存中的线性缓冲区，CPU 端把数据上传后，后续绘制尽量不重复传输，性能更稳定。EBO 用于索引绘制，核心价值是复用顶点：例如矩形两个三角形如果直接展开要写 6 个顶点，使用索引只需 4 个顶点 + 6 个索引，能减少内存占用和带宽压力，也便于网格管理。glDrawElements 就是利用 EBO 按索引组装图元，这也是我们代码里主绘制路径采用的方式。

VAO 经常最容易被误解。它不是顶点数据本身，而是与顶点读取规则息息相关：包括某个 attribute 从哪个 buffer 取、步长是多少、偏移是多少、类型是什么、是否启用该 attribute，以及与之关联的 element buffer 绑定关系。也就是说，VAO 把顶点格式描述固化下来，切换模型或材质时只需要切换 VAO，不必每帧重复调用大量 `glVertexAttribPointer`/`glEnableVertexAttribArray`。这对工程化渲染器非常关键，因为它把状态管理和绘制调用分离，提高了代码可维护性。

从渲染管线角度看，这三者的关系可以概括为：VBO/EBO 负责数据内容，VAO 负责数据解释方式。如果把顶点看成一本书，VBO 是正文，EBO 是目录，VAO 是阅读规则（每页从哪读、按什么字段读）。实验中常见错误也基本围绕三者关系：比如 attribute 步长/偏移写错会导致模型问题，EBO 未与当前 VAO 正确关联会导致 glDrawElements 结果异常，或者 VAO 切换时忘记对应 shader 的输入 layout。这次实验能够完成多种着色与阴影流程，底层正是依赖这套 VAO/VBO/EBO 机制把几何数据稳定送入 GPU。

2) 说明你实现了哪些渲染算法，并简述其思想。

在本次实验中，我实现并串联了一个较完整的实时渲染流程，核心算法可以概括为：几何变换、光照着色、环境映射、法线扰动、阴影映射与后期混合。它们并不是孤立功能，而是围绕同一条图形管线逐步叠加得到的。

三维变换

首先是**三维变换算法**。我实现了透视投影矩阵与 LookAt 观察矩阵，使模型从世界空间正确映射到屏幕空间。透视投影部分基于 FOV、宽高比、近远平面构造矩阵，并额外支持 `reverseZ` 分支以适配不同深度映射策略；观察矩阵部分通过 `forward / side / up` 构建正交基并加入平移项，实现相机视角控制。该算法的思想是把场景几何转换为相机可见图像，是后续所有光照与材质计算的空间基础。

本实验中，三维变换系统由相机类 `Camera` 实现，核心是 `projectionMatrix()` 与 `viewMatrix()` 两个函数。其作用把模型从世界空间逐步变换到裁剪空间，从而完成 3D 场景到 2D 屏幕的映射。实现思路可分为两部分：

首先在透视投影中，我们使用视场角 `fov`、宽高比 `aspect`、近远平面 `near/far` 构造标准透视矩阵。代码中先计算 `tanHalfFovInverse = 1 / tan(fov/2)`，再写入矩阵对角线项控制 X/Y 方向投影强度，这决定了透视“张角”和画面拉伸比例。Z 方向上，项目支持 `reverseZ`，因此按开关写入两套深度映射公式：普通模式下近处深度更小，反向模式下近处深度更大（有利于深度精度分布）。

其次在观察矩阵（LookAt）中，我们用相机位置 `eye`、观察目标 `center`、上方向 `up` 构造相机坐标系正交基：

```
forward = normalize(center - eye),
side = normalize(cross(forward, up)),
up = cross(side, forward)。
```

然后将 `side / up / -forward` 写入矩阵旋转部分，再通过与 `eye` 的点积写入平移项，实现“将世界坐标变换到相机视角空间”。这一步保证了相机移动、旋转后，场景观察结果正确变化。

这套实现的工程意义是：为后续 Blinn-Phong、PBR、阴影映射等算法提供统一且正确的空间变换基础。如果投影或视图矩阵错误，后面的光照、阴影、天空盒都会表现异常（例如拉伸、反向、抖动或裁剪错误）。

核心代码如下：

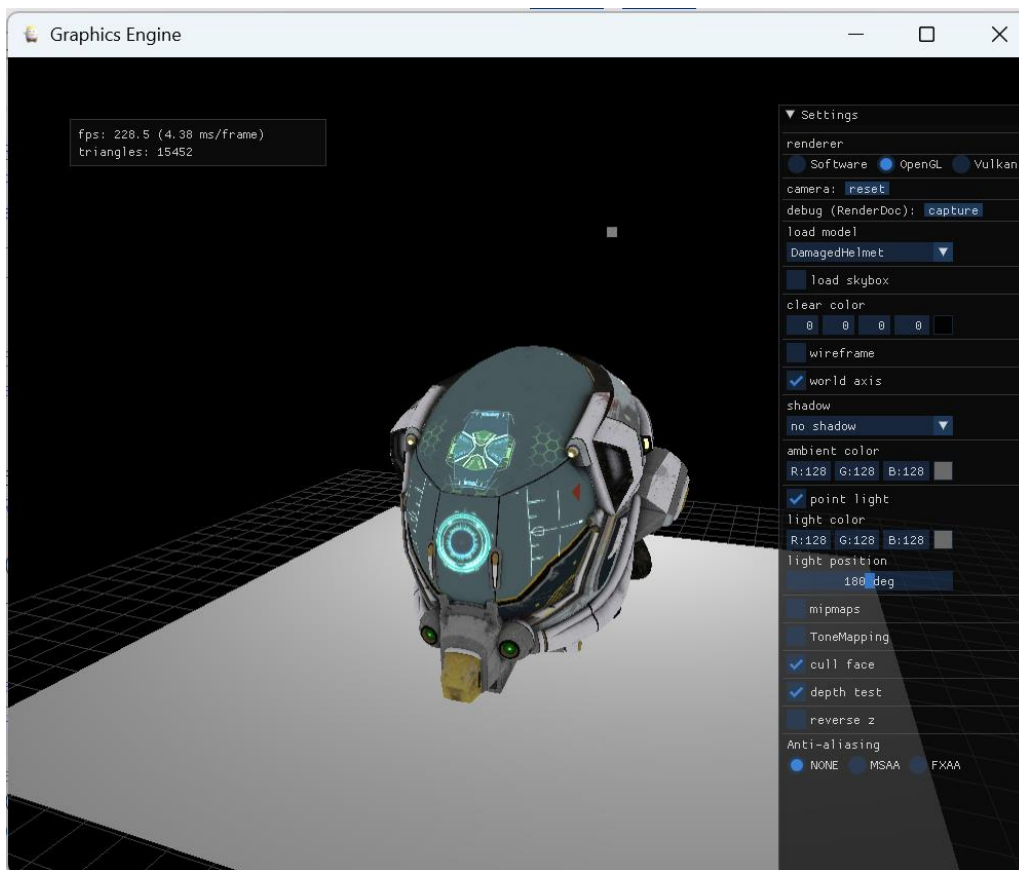
```
1. // src/Viewer/Camera.cpp
2. glm::mat4 Camera::projectionMatrix() const {
```

```

3.  glm::mat4 projection(0.f);
4.
5.  const float tanHalfFovInverse = 1.0f / std::tan(fov_ * 0.5f);
6.  projection[0][0] = tanHalfFovInverse / aspect_;
7.  projection[1][1] = tanHalfFovInverse;
8.  projection[2][3] = -1.0f;
9.
10. if (reverseZ_) {
11.     projection[2][2] = (near_ + far_) / (far_ - near_);
12.     projection[3][2] = (2.0f * near_ * far_) / (far_ - near_);
13. } else {
14.     projection[2][2] = (far_ + near_) / (near_ - far_);
15.     projection[3][2] = (2.0f * near_ * far_) / (near_ - far_);
16. }
17.
18. return projection;
19. }
20.
21. glm::mat4 Camera::viewMatrix() const {
22.     glm::vec3 forward(glm::normalize(center_ - eye_));
23.     glm::vec3 side(glm::normalize(cross(forward, up_)));
24.     glm::vec3 up(glm::cross(side, forward));
25.
26.     glm::mat4 view(1.f);
27.     view[0][0] = side.x;    view[1][0] = side.y;    view[2][0] = side.z;
28.     view[3][0] = -glm::dot(side, eye_);
29.
30.     view[0][1] = up.x;      view[1][1] = up.y;      view[2][1] = up.z;
31.     view[3][1] = -glm::dot(up, eye_);
32.
33.     view[0][2] = -forward.x; view[1][2] = -forward.y; view[2][2] = -forward.z;
34.     view[3][2] = glm::dot(forward, eye_);
35.
36.     return view;
37. }

```

运行截图如下，可见三维场景已经正确从相机视角显示：



Phong 着色模型

其次是局部光照明着色算法（Blinn-Phong）。我在片段着色器中实现了环境光、漫反射、镜面反射三部分：漫反射使用 $\max(\text{dot}(\mathbf{N}, \mathbf{L}), 0)$ ，镜面反射采用半角向量的 Blinn-Phong 形式 $\text{pow}(\max(\text{dot}(\mathbf{N}, \mathbf{H}), 0), \text{shininess})$ 。同时加入了点光源距离衰减，避免远处物体亮度不合理。该算法的思想是用经验模型近似真实光照，计算成本低、视觉反馈稳定，适合作为基础实时着色方案。

本实验中，Blinn-Phong 光照实现位于 `BlinnPhongGLSL.frag`，先通过法线（包含法线贴图分支）得到单位法线 $\mathbf{N}$ ，再构造光照方向 $\mathbf{L} = \text{normalize}(\mathbf{v_lightDirection})$ 与视线方向 $\mathbf{V} = \text{normalize}(\mathbf{v_cameraDirection})$ 。在漫反射项中，使用 Lambert 形式 $\max(\text{dot}(\mathbf{N}, \mathbf{L}), 0)$ ，保证背光面不产生负亮度；镜面项采用 Blinn-Phong 改良形式，先算半角向量 $\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V})$ ，再用 $\text{pow}(\max(\text{dot}(\mathbf{N}, \mathbf{H}), 0), \text{specularExponent})$ 控制高光集中程度。相比传统 Phong 反射向量法，Blinn-Phong 在实时渲染里更稳定，也更常用。

此外，代码中加入了点光源衰减：通过光源方向长度构造衰减因子 Attenuation，再得到 $\text{radiance} = \mathbf{u_pointLightColor} * \text{attenuation}$ ，使远处亮度自然减弱。最后，当开启阴影时，调用 `ShadowCalculation(...)` 得到阴影系数 shadow，对漫反射和镜面反射做 $(1.0 - \text{shadow})$ 调制。这样可避免“阴影区仍然高亮”的不合理现象，同时保留环境光作为基础亮度。

整体思想可以概括为：**局部光照**（Ambient + Diffuse + Specular）负责材质体积感，**衰减**负责空间真实感，**阴影调制**负责遮挡真实性。这三者叠加后，画面会提升到有立体和明暗层次。（也可在上面的截图中看到模型的视觉效果的确如此）

核心代码如下：

```
1. // src/Viewer/Shader/GLSL/BlinnPhongGLSL.frag (main 片段)
2. vec3 ambientColor = baseColor.rgb * u_ambientColor * ao;
3. vec3 diffuseColor = vec3(0.f);
4. vec3 specularColor = vec3(0.f);
5.
6. if (u_enableLight) {
7.     vec3 lightDirection = normalize(v_lightDirection);
8.     vec3 viewDirection = normalize(v_cameraDirection);
9.     vec3 halfDirection = normalize(lightDirection + viewDirection);
10.
11.     float NdotL = max(dot(N, lightDirection), 0.0f);
12.     float NdotH = max(dot(N, halfDirection), 0.0f);
13.
14.     vec3 lDir = v_lightDirection * pointLightRangeInverse;
15.     float attenuation = clamp(1.0f - dot(lDir, lDir), 0.0f, 1.0f);
16.     vec3 radiance = u_pointLightColor * attenuation;
17.
18.     diffuseColor = baseColor.rgb * radiance * NdotL;
19.     specularColor = radiance * (u_kSpecular * pow(NdotH, specularExponent));
20.
21.     if (u_enableShadow) {
22.         float shadow = ShadowCalculation(v_shadowFragPos, N);
23.         diffuseColor *= (1.0f - shadow);
24.         specularColor *= (1.0f - shadow);
25.     }
26. }
```

天空盒渲染

然后是**环境映射与天空盒算法**。我实现了天空盒顶点深度处理（常规 `pos.xyww`，`reverseZ` 时强制深度到远端）以及片段阶段的双分支采样：对全景图使用经纬度映射（`atan/asin` 将方向向量映射到 2D UV），对立方体贴图直接按方向采样。该算法的思想是用远景环境贴图替代真实无限场景，低成本获得背景与环境反射基础信息。

本实验中，天空盒的实现分为顶点阶段与片段阶段两部分。顶点阶段的关键不是普通几何变换，天空盒应始终位于场景最远处，不能遮挡模型。为此在 `SkyboxGLSL.vert` 中使用了特殊输出方式：普通深度模式下采用 `gl_Position = pos.xyww`，让 `z/w` 固定到远平面；在 `reverseZ` 开启时则将深度固定到 `0.0` 对应的远端。这种做法可避免天空盒与场景几何产生深度竞争，同时保留正确的视角变化（旋转相机时天空随视角转动）。

片段阶段的目标是从方向向量采样环境颜色，支持两种纹理来源。若是全景图（Equirectangular），先将方向向量归一化，再通过 `atan + asin` 做经纬度映射得到 UV；若是立方体贴图（CubeMap），则可直接以方向向量作为采样方向。你当前 `SkyboxGLSL.frag` 已完整实现这两条分支逻辑，因此该算法兼具教学性与实用性：既能说明经纬度映射原理，也可直接用于立方体环境贴图流程。

从工程角度看，天空盒算法不仅是画出背景，还与 PBR/IBL 流程关联：环境图作为场景全局光照和反射信息来源，后续可用于辐照度图和预过滤反射图生成。因此该实验为后续更真实的材质表现打下了基础。

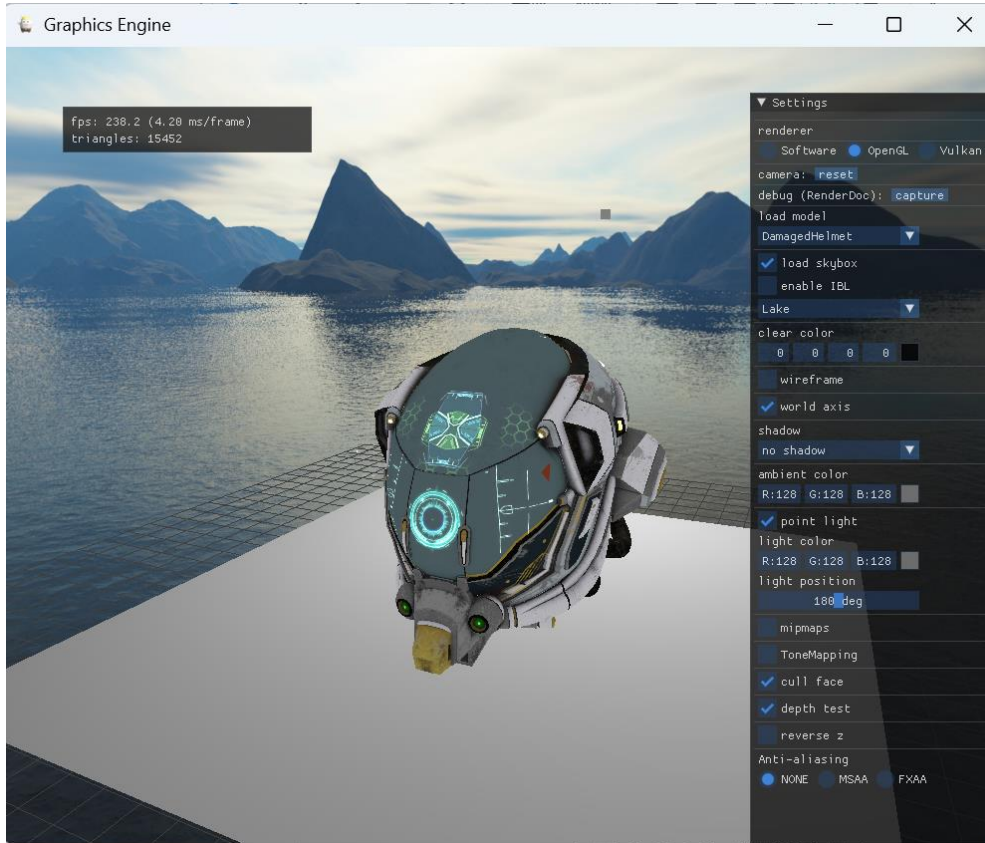
核心代码如下：

```
1. // src/Viewer/Shader/GLSL/SkyboxGLSL.vert
2. void main() {
3.     vec4 pos = u_modelViewProjectionMatrix * vec4(a_position, 1.0);
4.     v_worldPos = a_position;
5.
6.     if (u_reverseZ) {
7.         gl_Position = vec4(pos.x, pos.y, 0.0, pos.w);
8.     } else {
9.         gl_Position = pos.xyww;
10.    }
11. }
```

```
1. // src/Viewer/Shader/GLSL/SkyboxGLSL.frag
2. vec2 SampleSphericalMap(vec3 dir) {
3.     vec2 uv = vec2(atan(dir.z, dir.x), asin(-dir.y));
4.     uv *= vec2(0.1591f, 0.3183f);
5.     uv += 0.5f;
6.     return uv;
7. }
8.
9. void main() {
10.    #if defined(EQUIRECTANGULAR_MAP)
11.        vec2 uv = SampleSphericalMap(normalize(v_worldPos));
12.        FragColor = texture(u_equirectangularMap, uv);
13.    #else
```

```
14.     FragColor = texture(u_cubeMap, v_worldPos);
15.     #endif
16. }
```

天空盒开启后的截图如下，能明显感觉到环境背景包围感：



切线空间切法线贴图

在材质细节方面，我实现了切线空间法线贴图（TBN）。具体做法是：法线贴图采样值从 $[0, 1]$ 还原到 $[-1, 1]$ ，对 T/N 做归一化并用 Gram-Schmidt 正交化，再通过 TBN 矩阵把切线空间法线变换到世界空间。该算法思想是“用纹理扰动法线而非增加几何面数”，在不显著增加顶点负担的前提下提升表面细节表现。

本实验中，法线贴图采用切线空间方案，核心目的是在不增加几何面数的前提下提升表面细节。实现位置在 `PbrGLSL.frag` 的 `GetNormalFromMap()`。其思想是先在纹理空间读取“扰动法线”，再通过 TBN 矩阵把它变换到世界空间，使后续光照计算 $(N \cdot L, N \cdot H)$ 能正确反映凹凸细节。

具体实现流程如下：首先从法线贴图采样 `tangentNormal = texture(...).xyz`，并将其从 $[0, 1]$ 映射回 $[-1, 1]$ ；随后读取并归一化顶点法线 N 与切线 T 。为了避免切线与法线不正交导致的偏斜误差，代码使用 Gram-Schmidt 进行正交化： $T = \text{normalize}(T - \text{dot}(T, N) * N)$ 。接着通过叉

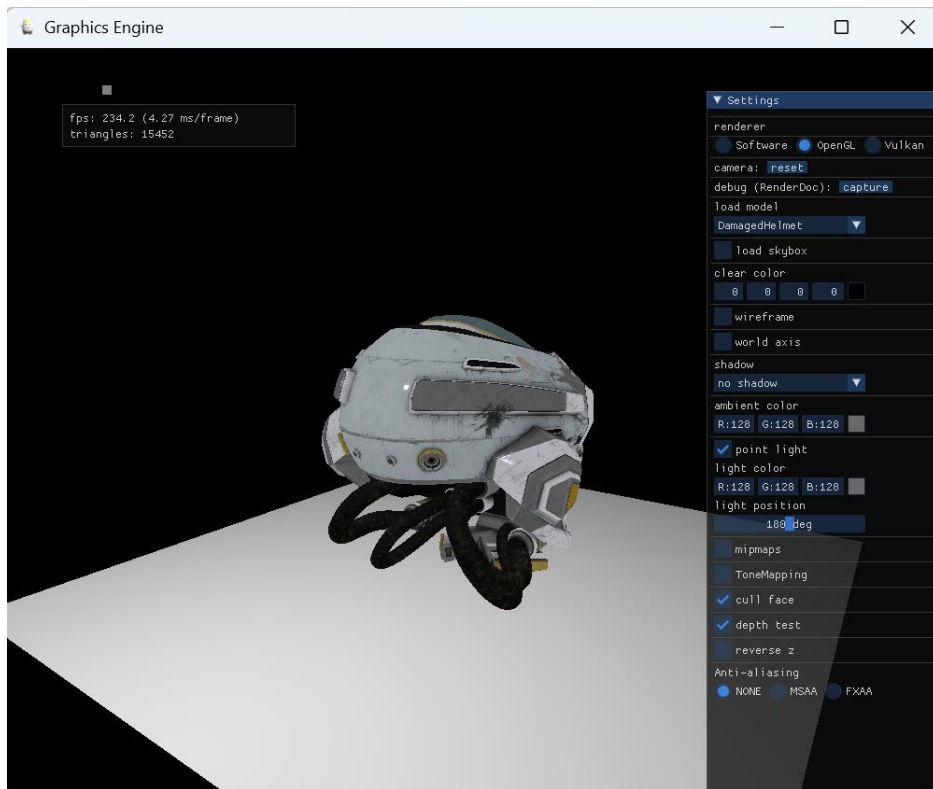
积得到副切线 B，构造 $\text{mat3 TBN} = \text{mat3}(\text{T}, \text{B}, \text{N})$ ，最后返回 $\text{normalize}(\text{TBN} * \text{tangentNormal})$ 作为世界空间法线参与光照。该流程保证了法线贴图在模型表面方向变化时依然保持正确的细节朝向。

该算法算法的价值在于用纹理表达几何细节。与真实细分网格相比，它几乎不增加顶点处理开销，却能显著增强材质真实感（如石墙、金属拉丝、皮革纹理等）。在本次实验链路中，它与 PBR/Blinn-Phong 都是直接耦合的：光照模型本身没变，但输入法线更精细，因此最终视觉质量明显提升。

核心代码如下：

```
1. // src/Viewer/Shader/GLSL/PbrGLSL.frag
2. vec3 GetNormalFromMap() {
3.     #if defined(NORMAL_MAP)
4.         vec3 tangentNormal = texture(u_normalMap, v_texCoord).xyz * 2.0f -
            1.0f;
5.
6.         vec3 N = normalize(v_normal);
7.         vec3 T = normalize(v_tangent);
8.
9.         // Gram-Schmidt orthogonalization
10.        T = normalize(T - dot(T, N) * N);
11.
12.        vec3 B = normalize(cross(N, T));
13.        mat3 TBN = mat3(T, B, N);
14.
15.        return normalize(TBN * tangentNormal);
16.    #else
17.        return normalize(v_normalVector);
18.    #endif
19. }
```

打开点光源后模型截图如下，可见物体表面会出现细节凹凸高光变化，旋转光源/视角时，细节明暗会跟着变化。



阴影映射

阴影部分实现了**阴影映射 + PCF 软阴影**。流程包括：裁剪空间坐标透视除法、OpenGL 深度区间 $[-1, 1] \rightarrow [0, 1]$ 重映射、当前深度与阴影图深度比较（并兼容 reverseZ 判定方向），以及 3×3 邻域 PCF 平均得到更平滑的阴影边缘。核心思想是从光源视角把当前片元投影到阴影贴图中，拿当前深度与贴图深度比较，若被更近深度挡住则判为阴影。

第一步是坐标变换。代码先做透视除法 $\text{projCoords} = \text{fragPos.xyz} / \text{fragPos.w}$ ，得到 NDC 深度，再将 OpenGL 的 $[-1, 1]$ 深度区间映射到 $[0, 1]$ （ $\text{currentDepth} = \text{currentDepth} * 0.5 + 0.5$ ）。这样才能和阴影贴图中的深度值同域比较。边界上还保留了范围检查，防止采样越界导致异常闪烁。

第二步是基础阴影比较。采样 $\text{mapDepth} = \text{texture}(\text{u_shadowMap}, \text{projCoords.xy}).r$ 后，根据 reverseZ 选择比较方向：常规深度下使用“当前深度更大则在阴影中”，反向深度下方向相反。比较时加入 bias 以抑制阴影粉刺（self-shadow acne）。最终返回 shadow（0 或 1），在主光照里通过 $(1.0 - \text{shadow})$ 衰减漫反射和镜面项。

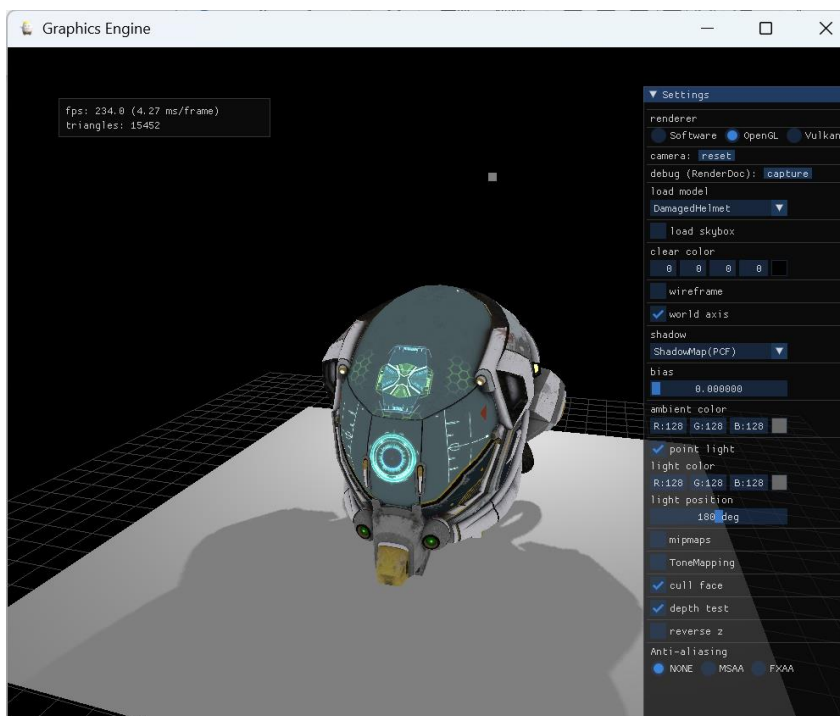
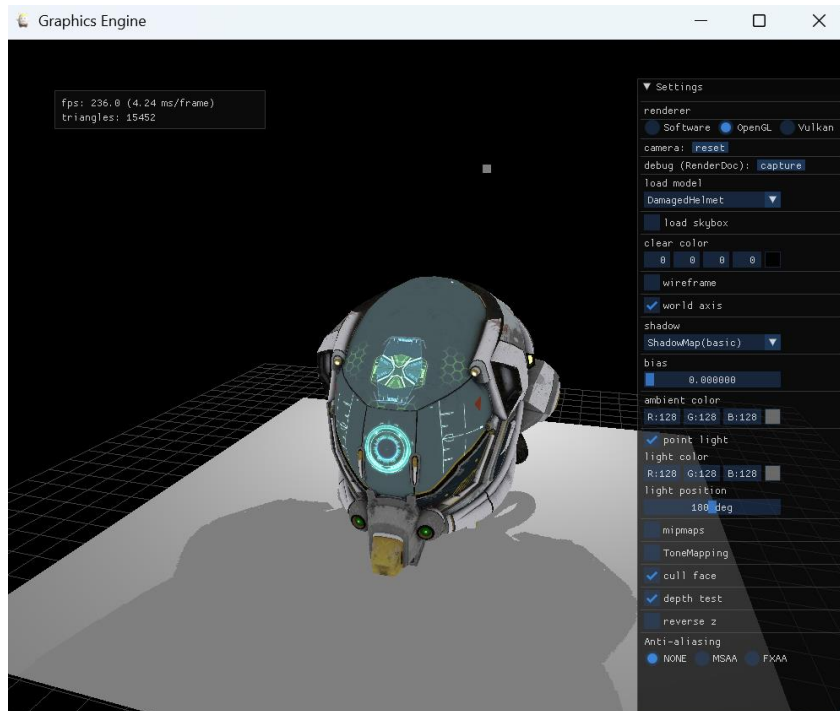
第三步是 PCF（百分比接近过滤）软阴影。PCF 分支通过 textureSize 求纹素步长，在 3×3 邻域做九次深度比较并平均。它的意义不是改变阴影几何正确性，而是降低硬边锯齿，让阴影边缘过渡更自然，视觉上更接近真实面积光效果。

核心代码如下：

```
1. // src/Viewer/Shader/GLSL/BlinnPhongGLSL.frag
2. float ShadowCalculation(vec4 fragPos, vec3 normal) {
3.     vec3 projCoords = fragPos.xyz / fragPos.w;
4.     float currentDepth = projCoords.z;
5.
6.     // OpenGL depth remap
7.     currentDepth = currentDepth * 0.5 + 0.5;
8.
9.     if (currentDepth < 0.0 || currentDepth > 1.0) {
10.         return 0.0;
11.     }
12.
13.     float bias = u_bias.x;
14.     float shadow = 0.0;
15.
16.     // basic shadow
17.     if (u_bias.y == 0) {
18.         float mapDepth = texture(u_shadowMap, projCoords.xy).r;
19.         if (u_reverseZ) {
20.             shadow = (currentDepth + bias < mapDepth) ? 1.0 : 0.0;
21.         } else {
22.             shadow = (currentDepth - bias > mapDepth) ? 1.0 : 0.0;
23.         }
24.     }
25.     // 3x3 PCF
26.     else if (u_bias.y == 1) {
27.         vec2 pixelOffset = 1.0 / vec2(textureSize(u_shadowMap, 0));
28.         for (int x = -1; x <= 1; ++x) {
29.             for (int y = -1; y <= 1; ++y) {
30.                 float mapDepth = texture(u_shadowMap, projCoords.xy +
                    vec2(x, y) * pixelOffset).r;
31.                 if (u_reverseZ) {
32.                     shadow += (currentDepth + bias < mapDepth) ? 1.0
                        : 0.0;
33.                 } else {
34.                     shadow += (currentDepth - bias > mapDepth) ? 1.0
                        : 0.0;
35.                 }
36.             }
37.         }
38.         shadow /= 9.0;
39.     }
```

```
40.  
41.     return shadow;  
42. }
```

下面展示的是基础阴影和 PCF 阴影的效果：



颜色混合

最后是颜色混合算法。我在渲染器状态设置中实现了 `glBlendEquationSeparate` 与 `glBlendFuncSeparate`，按渲染状态结构分别设置 RGB/Alpha 的混合方程和因子，并用统一状态管理接口启停 `GL_BLEND`。该算法思想是通过源颜色与目标颜色的加权组合完成透明、半透明与叠加效果，是多通道渲染结果合成的重要环节。

本实验中，颜色混合在渲染器状态设置阶段完成，位置是 `RendererOpenGL.cpp` 的 `setPipelineStates()`。实现重点不是“写死某个混合参数”，而是将混合状态抽象为 `blendParams`，再统一映射到 OpenGL API。这种做法使混合策略可配置、可复用，适合多材质与多渲染路径切换。

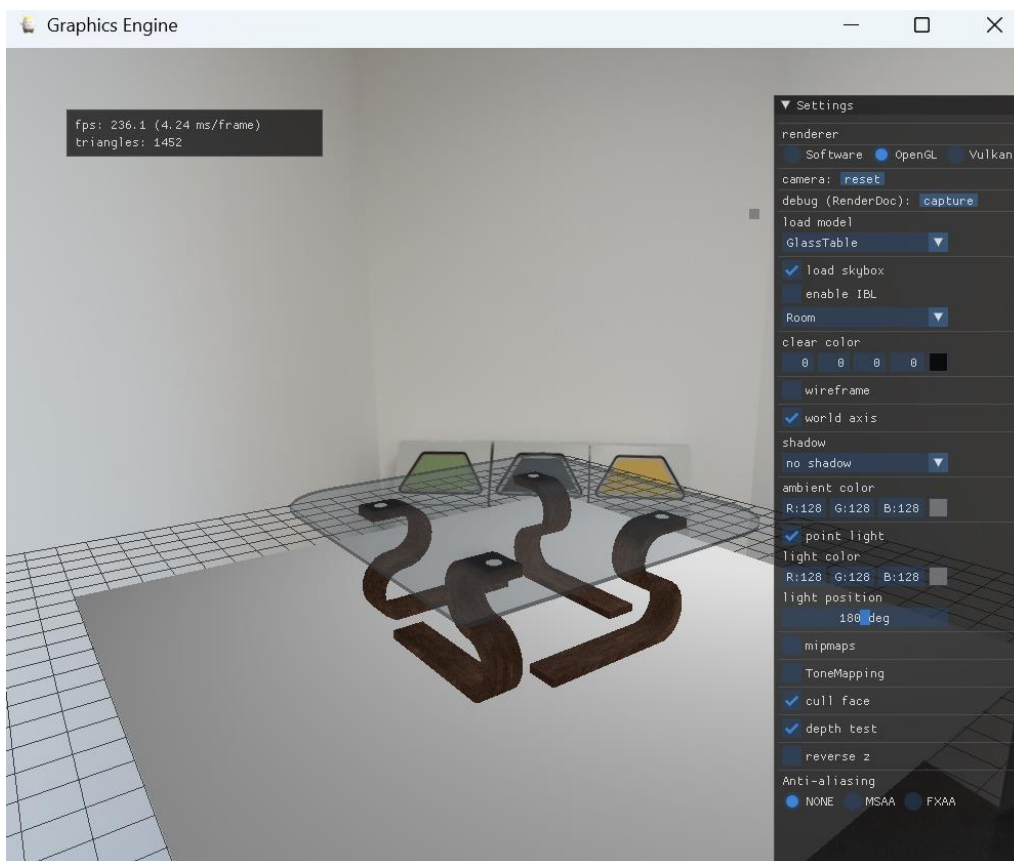
流程上，首先通过 `GL_STATE_SET(renderStates.blend, GL_BLEND)` 开启或关闭混合功能。随后调用 `glBlendEquationSeparate(...)` 分别设置 RGB 与 Alpha 的混合方程，再调用 `glBlendFuncSeparate(...)` 分别设置 RGB 与 Alpha 的源因子/目标因子。你当前实现里这些参数都来自 `blendParams`，并通过 `OpenGL::cvtBlendFunction/cvtBlendFactor` 完成枚举转换，符合“渲染状态驱动 OpenGL 状态机”的设计思想。

在公式层面，混合可写为： $C_{result} = C_{src} * F_{src} + C_{dst} * F_{dst}$ 。通过设置不同因子（如 `SRC_ALPHA` 与 `ONE_MINUS_SRC_ALPHA`），可以实现透明物体、半透明叠加、发光叠加等效果。实验中你们把混合与深度/剔除状态统一放在同一套 pipeline 状态管理中，这是工程化渲染器常见且正确的做法，能降低状态错配概率。

核心代码如下：

```
1. // src/Render/OpenGL/RendererOpenGL.cpp (setPipelineStates)
2. GL_STATE_SET(renderStates.blend, GL_BLEND)
3.
4. GL_CHECK(glBlendEquationSeparate(
5.     OpenGL::cvtBlendFunction(renderStates.blendParams.blendFuncRgb),
6.     OpenGL::cvtBlendFunction(renderStates.blendParams.blendFuncAlpha)))
7.     ;
8. GL_CHECK(glBlendFuncSeparate(
9.     OpenGL::cvtBlendFactor(renderStates.blendParams.blendSrcRgb),
10.    OpenGL::cvtBlendFactor(renderStates.blendParams.blendDstRgb),
11.    OpenGL::cvtBlendFactor(renderStates.blendParams.blendSrcAlpha),
12.    OpenGL::cvtBlendFactor(renderStates.blendParams.blendDstAlpha)));
```

下图展示了透明玻璃材质颜色混合和过渡更自然：



整体来看，本实验完成的算法体系覆盖了从相机变换到最终颜色输出的关键链路，既包括经典实时图形基础（变换、Phong、阴影、混合），也包含了现代材质表现中的核心技术点（环境映射、法线贴图、reverseZ 兼容）。

3) 在将多个图形算法结合时，需要注意什么问题？遇到了什么难点？

在将多个图形算法组合到同一渲染器时，最需要注意的是一致性和状态管理。一致性首先体现在空间与数据语义上：顶点阶段、片段阶段、阴影阶段要使用一致的坐标空间定义（模型/世界/观察/裁剪），否则会出现组合全错的问题。例如法线贴图要求切线空间到世界空间变换正确，阴影映射又要求光源空间投影与深度比较一致；一旦矩阵链或深度区间映射不统一，结果就是阴影漂浮、光照方向错误、法线细节反向等。其次是一致的深度策略，项目里同时支持普通深度与reverseZ，这会影响投影矩阵、深度清除值、深度比较函数、阴影比较方向、天空盒深度处理，必须成套修改，不能只改一处。

感谢老师课上的讲解和学长们实验的设计，本次实验在文档（以及 AI）的帮助下比较容易实现，但问题是可能学生在借用各种手段实现后对渲染器的理解还是不够深厚，对最终模型的效果也是似懂非懂。我觉得以后要进一步改良的话也许可以考虑把这套做成分层目标 + 可视化验收 + 纠错驱动，教学效果会明显更好。

第一，分层目标明确：每个任务分成“必做（跑通）—进阶（质量）—挑战（性能）”。比如阴影映射，必做是基础深度比较，进阶是 PCF，挑战是 bias 自适应与性能统计。这样不同基础的同学都能完成且有上升空间，避免两极分化。

第二，强制对比截图作为验收件。每个任务至少提交 **before/after** 两图（如开关法线贴图、基础阴影 vs PCF、开关天空盒），并附带详细的解释“改了什么、现象是什么、为什么会这样”。这比只交代码更能证明理解，也方便助教快速评分。

第三，安排统一故障排查实验。给几个经典错误（坐标系错、深度范围错、等），让学生按现象定位原因。这个环节对能力提升非常大，且和真实开发高度一致。

2.3 实验小结

这次实验最大的收获是从写单个 OpenGL API 提升到理解一条完整渲染管线怎么协同工作。从前期做窗口与上下文，到做相机投影与 LookAt 时，我真正理解了空间变换不是公式背诵，而是世界空间、观察空间、裁剪空间之间的一致性约束。尤其是引入 reverseZ 后，我体会到图形系统里很多模块是强耦合的：投影矩阵、深度清除值、深度比较函数、阴影比较方向都要同步，否则就会出现局部正确、整体错误的问题。在光照与材质部分，我对 Blinn-Phong、法线贴图(TBN)、天空盒、阴影映射和 PCF 的理解更具体了。与此同时，颜色混合实验让我理解了渲染状态机管理的重要性：不是简单开关，而是要把方程、因子、深度写入策略和绘制顺序一起考虑。

附录

任务 01 GLFW 窗口创建核心代码

```
1. // src/Viewer/Main.cpp
2. GLFWwindow* exp1() {
3.     GLFWwindow* window = nullptr;
4.     window = glfwCreateWindow(SCR_WIDTH, SCR_HEIGHT, "Graphics Engine",
5.                             nullptr, nullptr);
6.     if (!window) {
7.         return nullptr;
8.     }
9.     GLFWimage images[1];
10.    images[0].pixels = stbi_load("../icon.png", &images[0].width, &images[0].height, 0, 4);
11.    if (images[0].pixels) {
```

```
12.     glfwSetWindowIcon(window, 1, images);
13.     stbi_image_free(images[0].pixels);
14. } else {
15.     LOGE("Failed to load icon: ../icon.png");
16. }
17.
18.     return window;
19. }
```